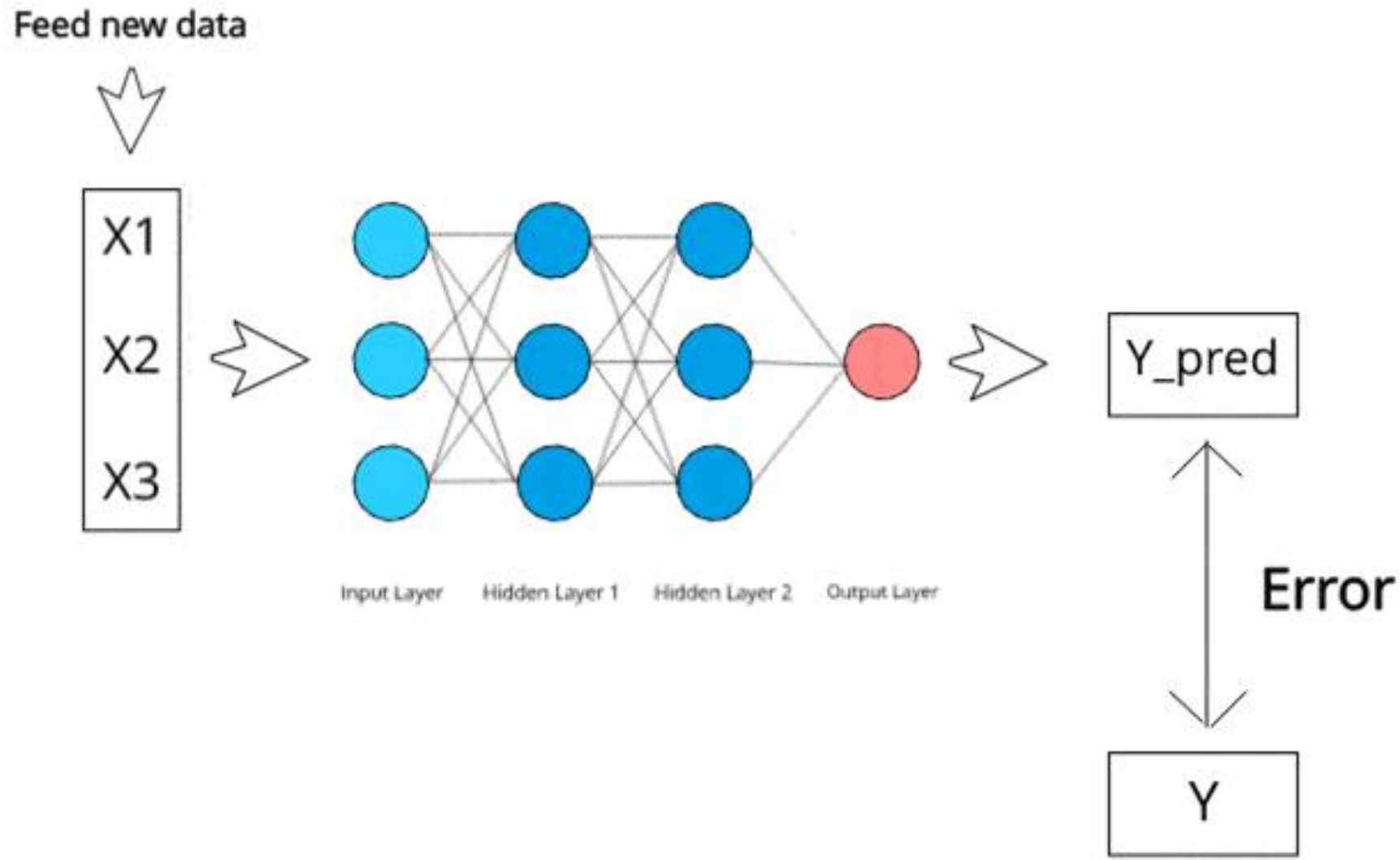
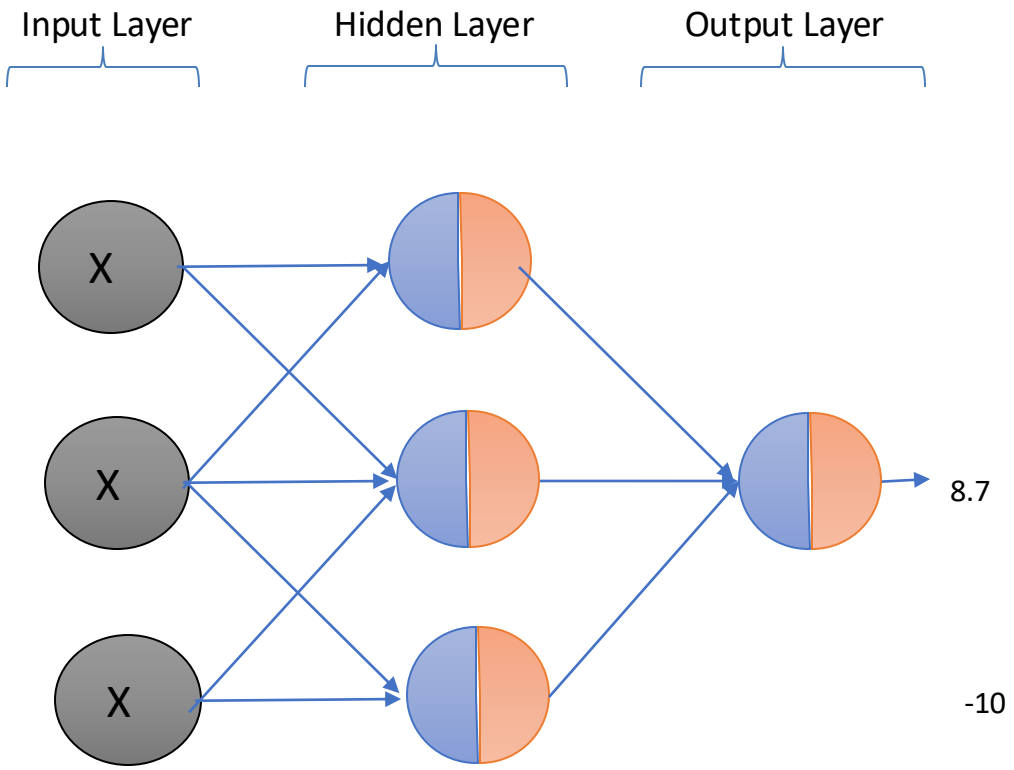
Deep Learning Course

Improving Performance

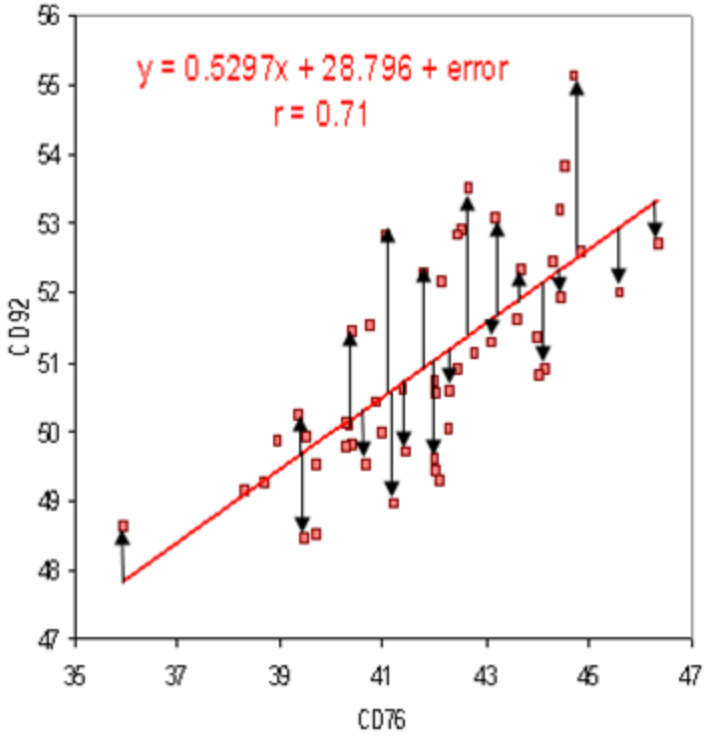
Recap


Feed new data
X1
X2
X3
Input Layer
Hidden Layer 1
Hidden Layer 2
Output Layer
Y_pred
Error
Y

Calculating Errors – Real Valued Outputs

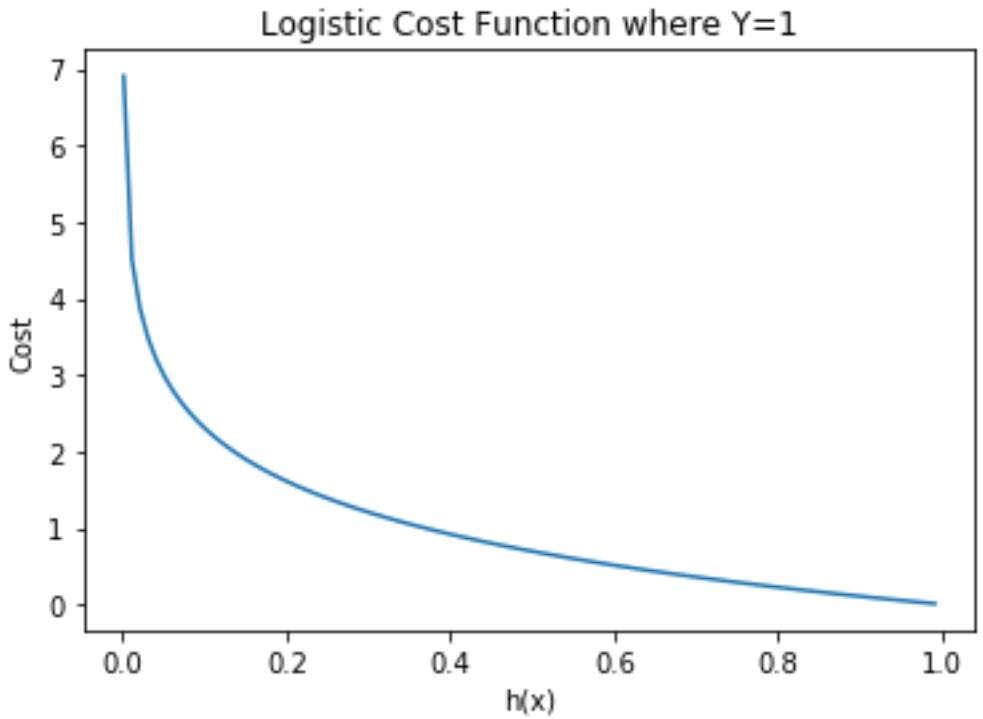


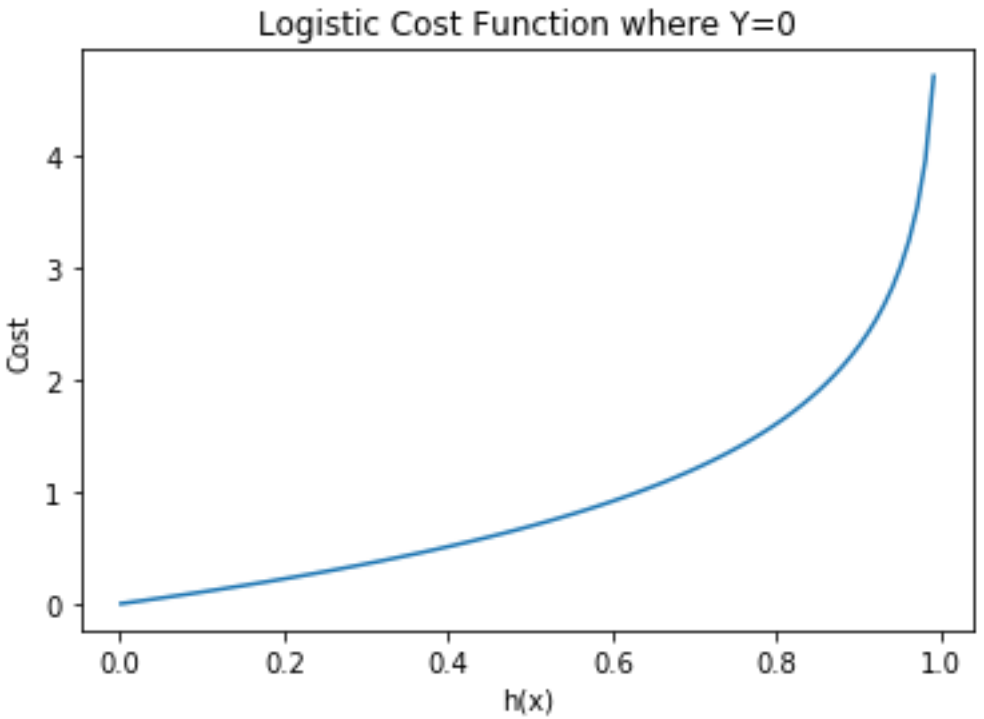
$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \tilde{y}_i)^2$$



Cross Entropy Loss Function

$$cost(h_\theta(x^{(i)}), y^{(i)}) = -y^{(i)} log(h_\theta(x^{(i)})) - (1 - y) log(1 - h_\theta(x^{(i)}))$$



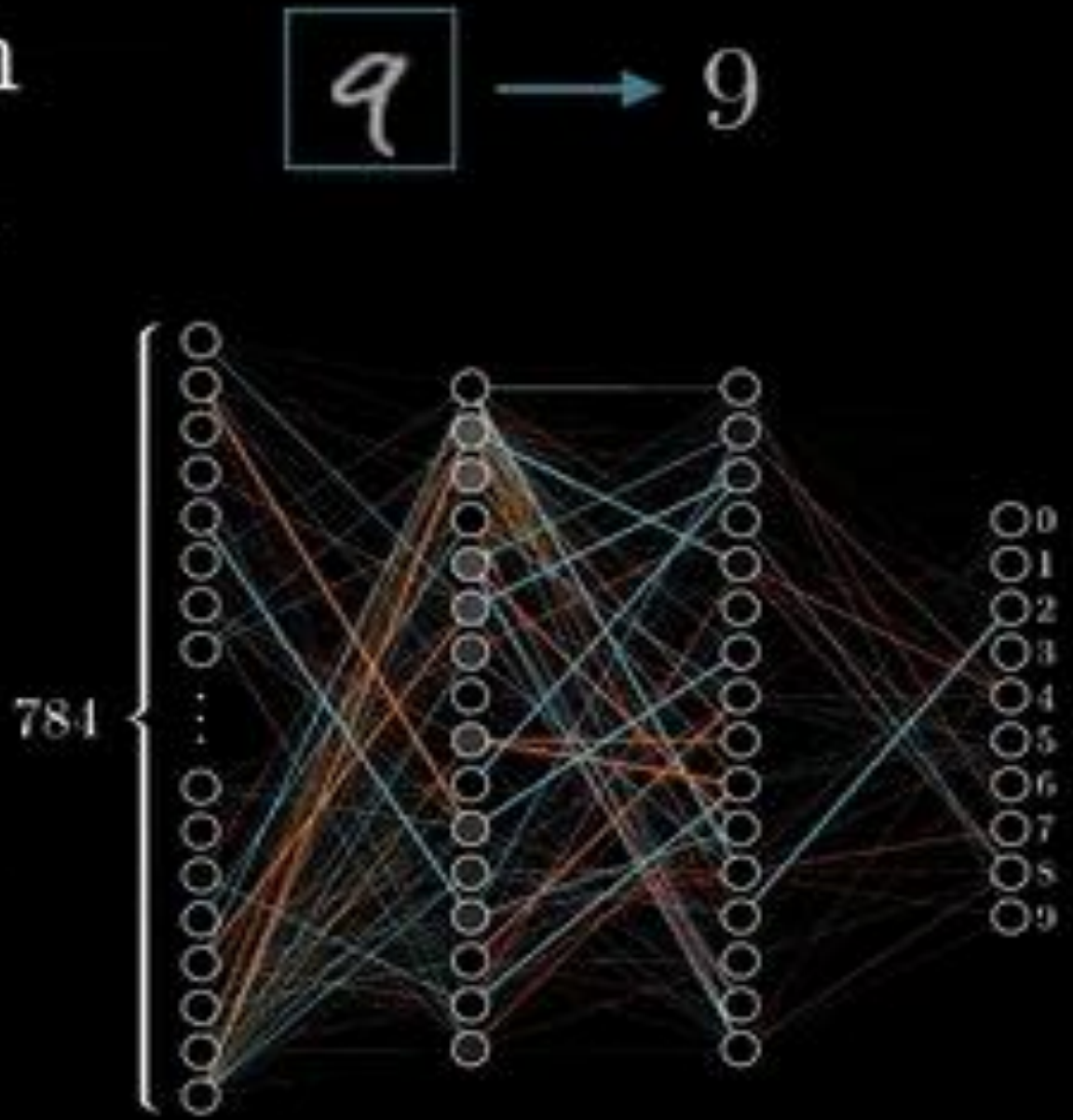


Backpropagation Algorithm

For each instance i in the training data:

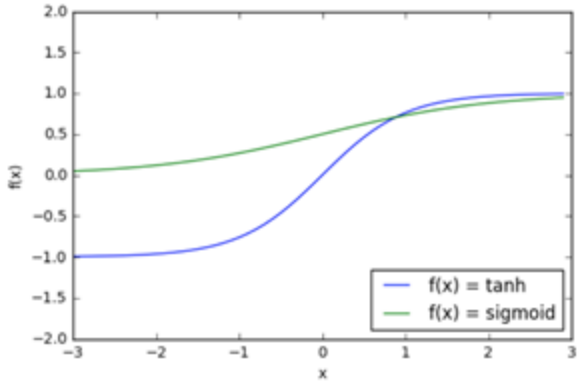
> Forward Propagate $x[i]$ through the network
>
> Backpropagate Error Derivatives back through the network
>
> > Calculate error derivative for final layer using $\delta = (a - y)\ (a(1 - a))$ for $y[i]$ $\quad \delta = (a - y)\ (a(1 - a))$ for $y[i]$
> >
> > Calculate error derivative for hidden layers using $\delta^l = \sum_j (\delta^{l+1}\ \theta_i^l)\ g'(z^l)$ $\quad \delta^l = \sum_j (\delta^{l+1}\ \theta_i^l)\ g'(z^l)$
> >
> > For each hidden layer accrue weight updates $\Delta_{ij}^{(l)} = \Delta_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$ $\quad \Delta_{ij}^{(l)} = \Delta_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l+1)}$
>
> Update weights with accrued weight changes $\theta = \theta + \Delta$ $\quad \theta = \theta + \Delta$

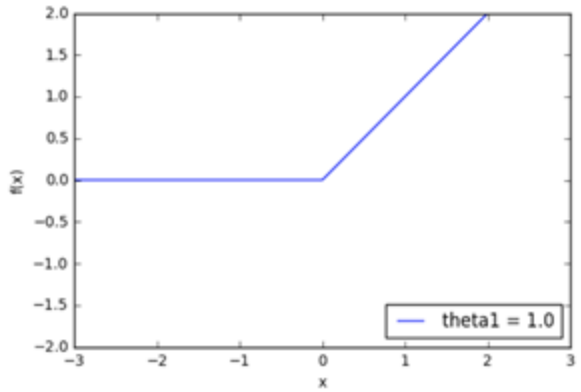
Training in progress. . .



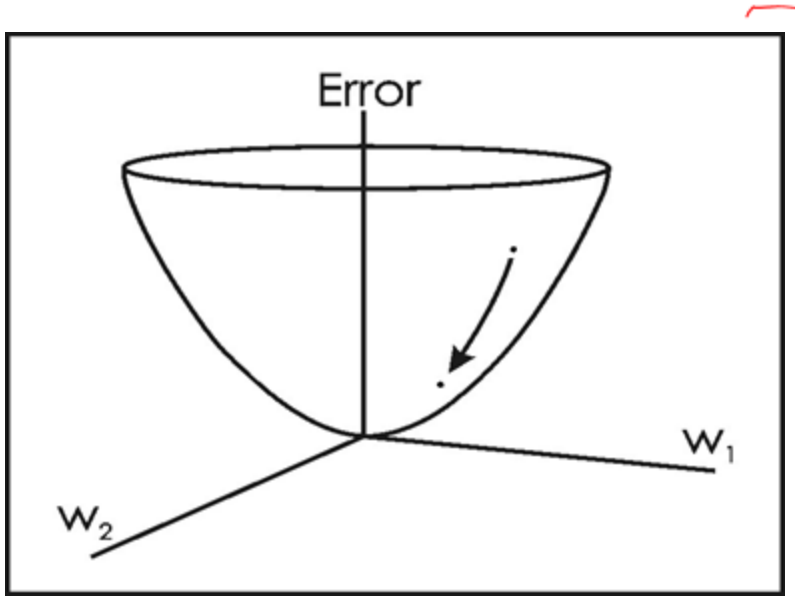
https://www.google.com/search?q=backpropagation&tbm=isch&tbs=itp%3Aanimated&hl=en&ved=0CAMQpwVqFwoTCIDx0LXVsOcCFQAAAAAdAAAAABAC&biw=1213&bih=608#imgrc=CufDRhQcARzdoM

Today





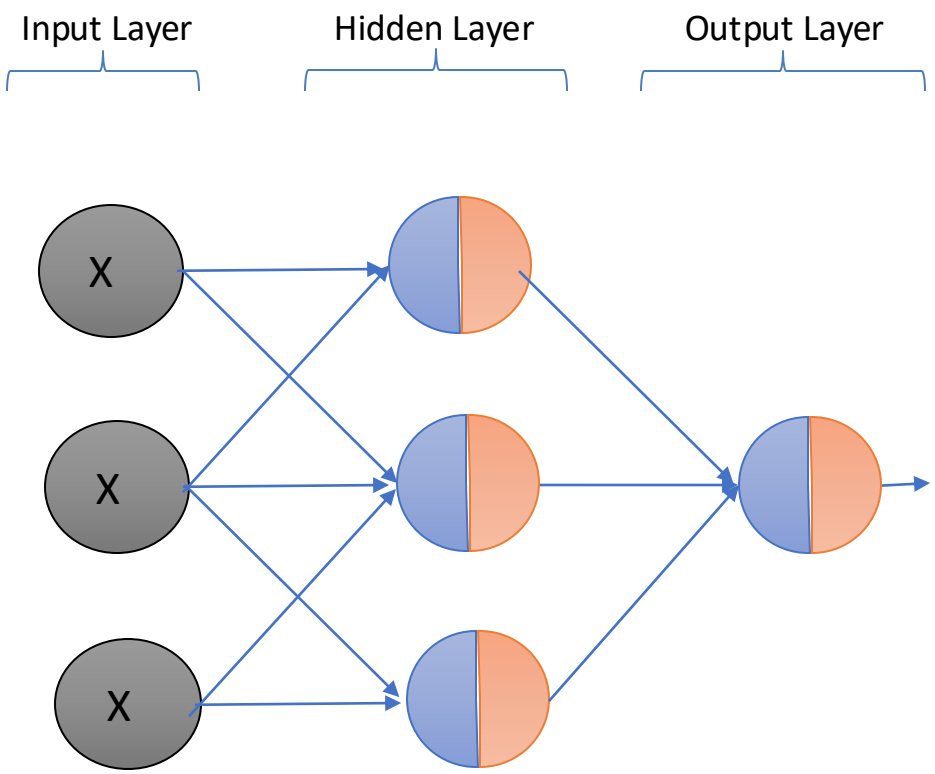
Alternative Application Functions



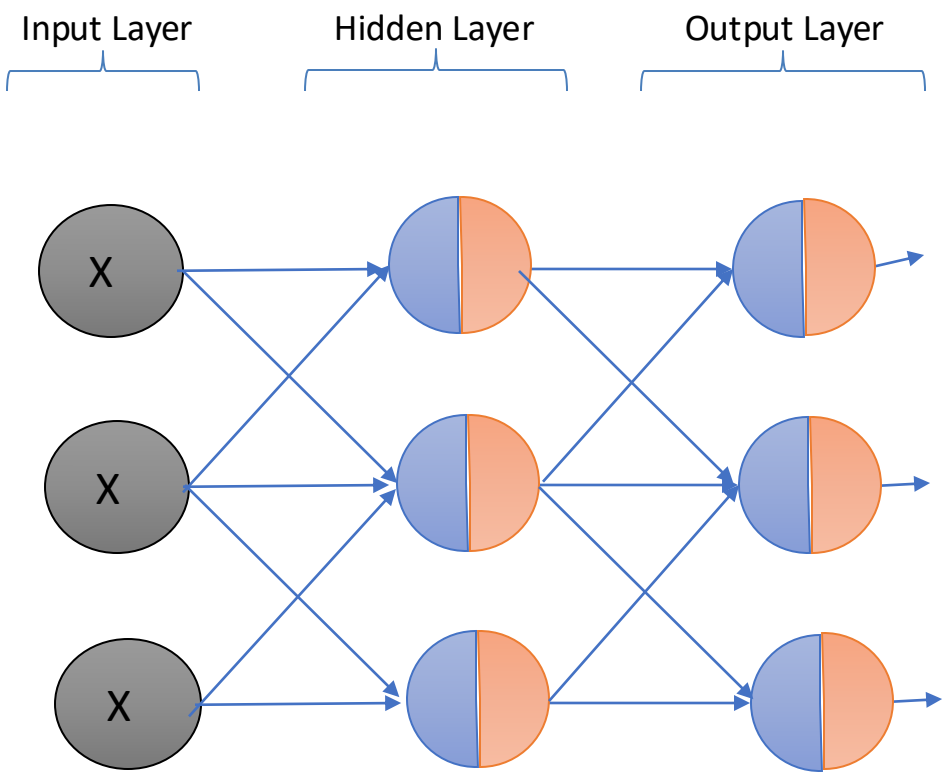
Optimizers for Training

Alternative Activations

From outputs with single units!



Errors in Outputs with multiple Units!

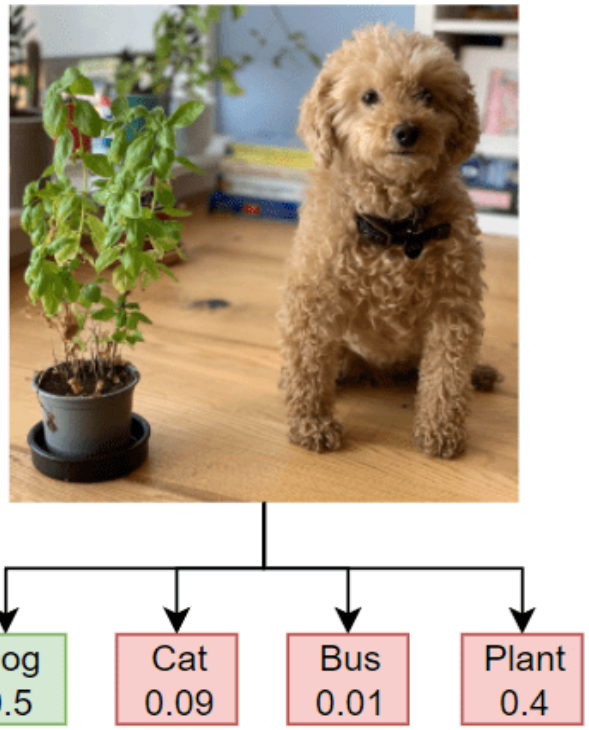


Errors in Outputs with multiple Units!

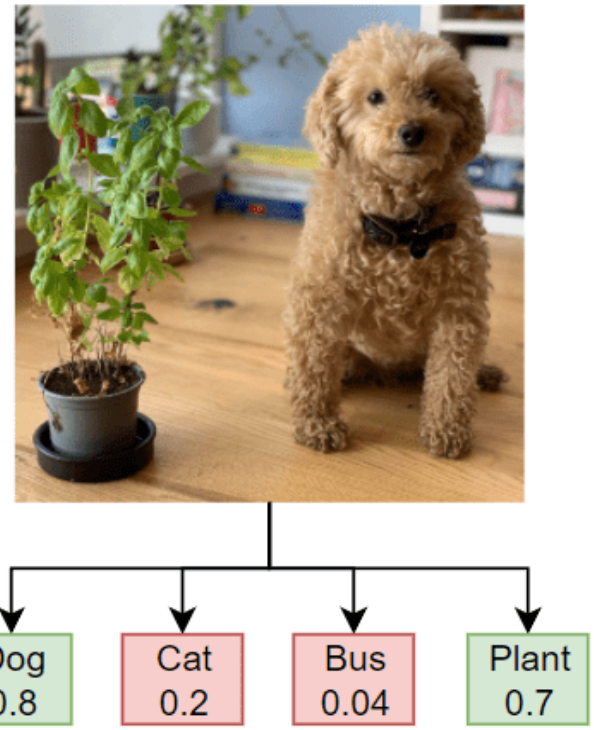
Binary Classification



Multiclass Classification

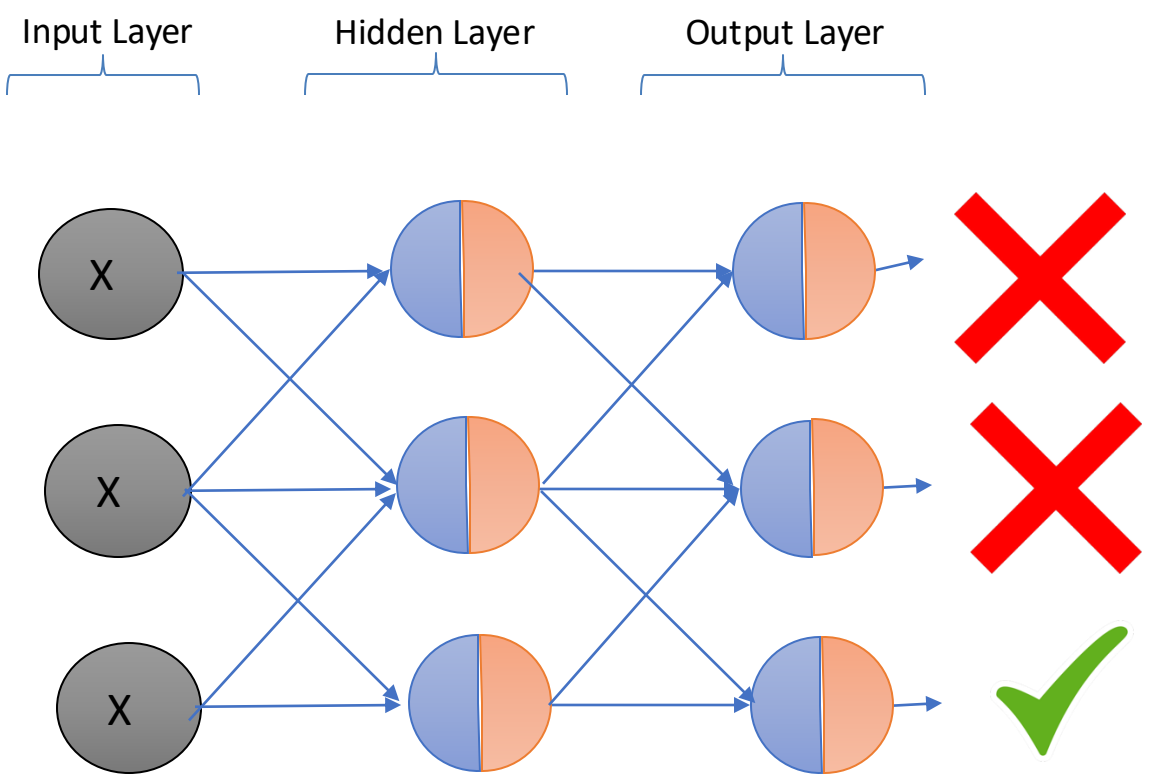


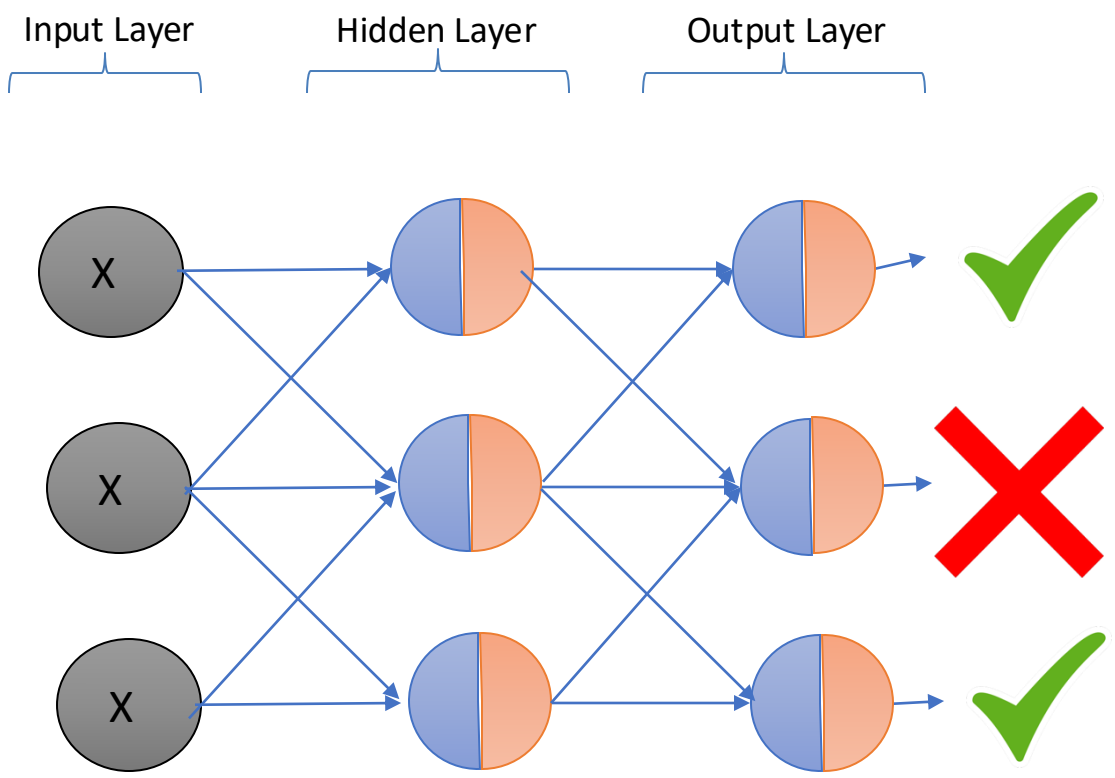
Multilabel Classification



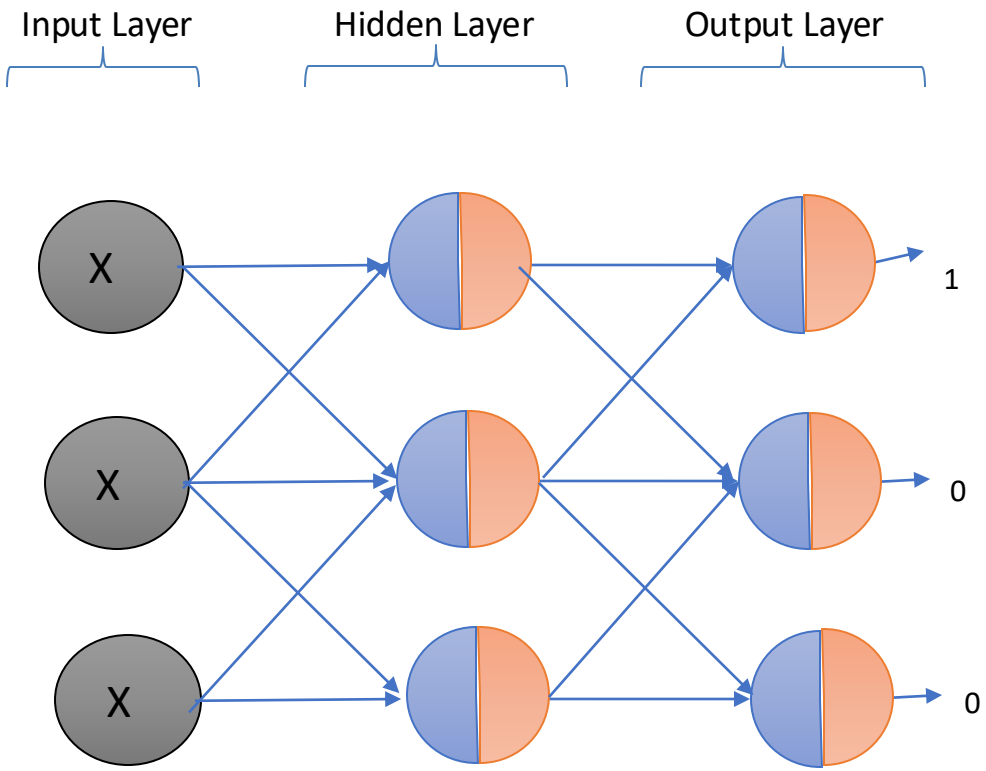
https://www.mathworks.com/help/deeplearning/ug/multilabel-image-classification-using-deep-learning.html

Multiclass (Multinomial) vs Multilabel





Calculating Errors – Multi-Label Outputs



Loss Function – Cross Entropy Loss Function

$$cost_{CELF}(a, y) = -y\ log(a) + (1 - y)log(1 - a)$$

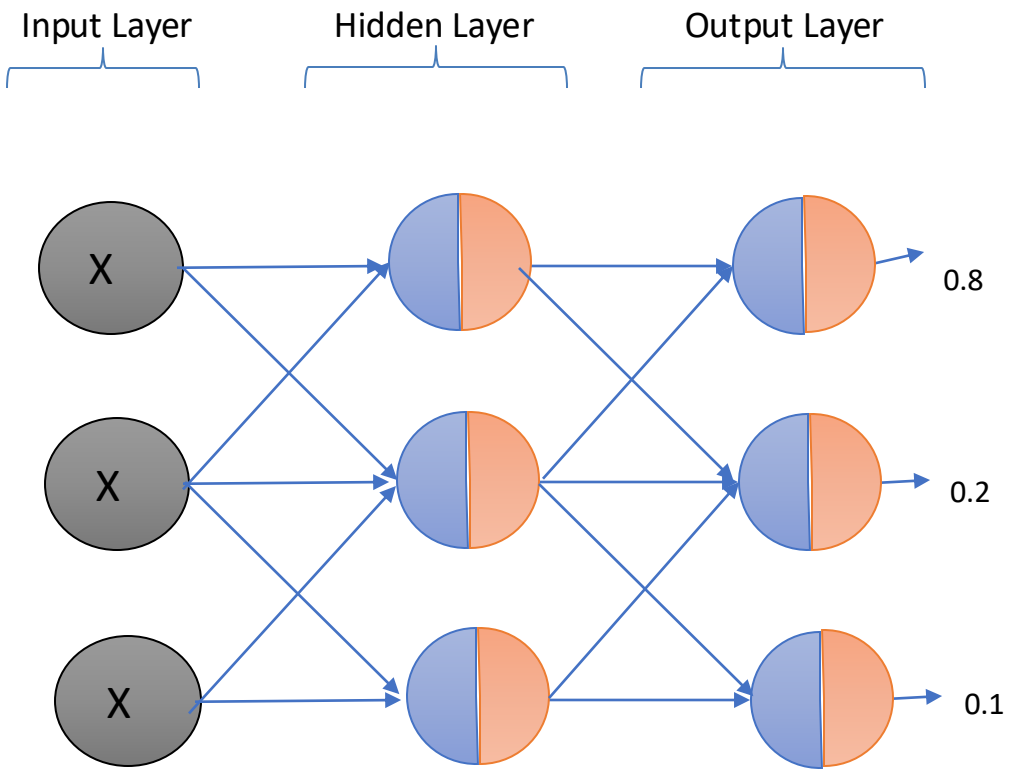
Error Derivative (final Layer)

$$\frac{\partial E}{\partial \theta_i^{l-1}} = (y - a)\ a_i^{l-1}$$

Total Loss =
Averaged loss over all output nodes

Error derivative is node by node.

Calculating Errors – Multinomial Outputs



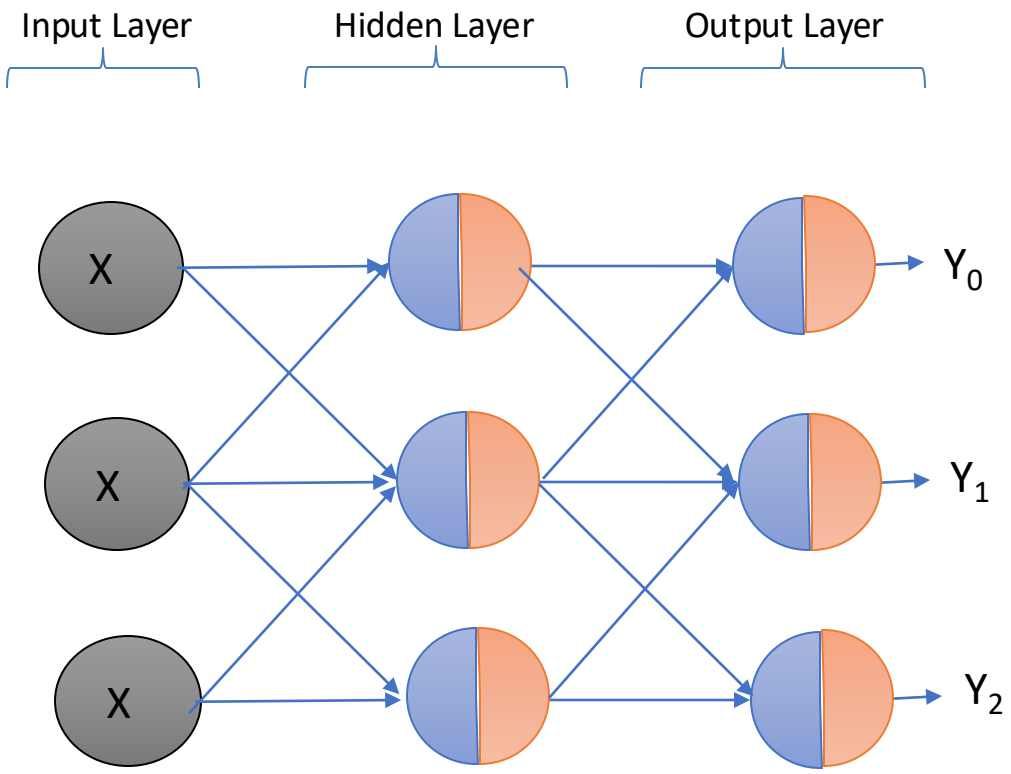
Remember / Assumption – One positive outcome per instance. Everything else negative, e.g., figure out the type of animal in an image.

Loss Function – Cross Entropy Loss Function

$$cost_{CELF}(a, y) = -y\ log(a) + (1 - y)log(1 - a)$$

We can use this but it isn't great.

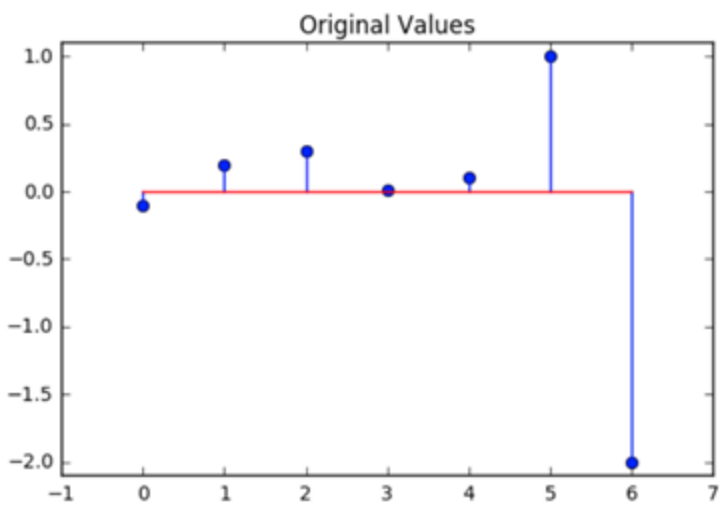
Multinomial: Softmax Activation Function

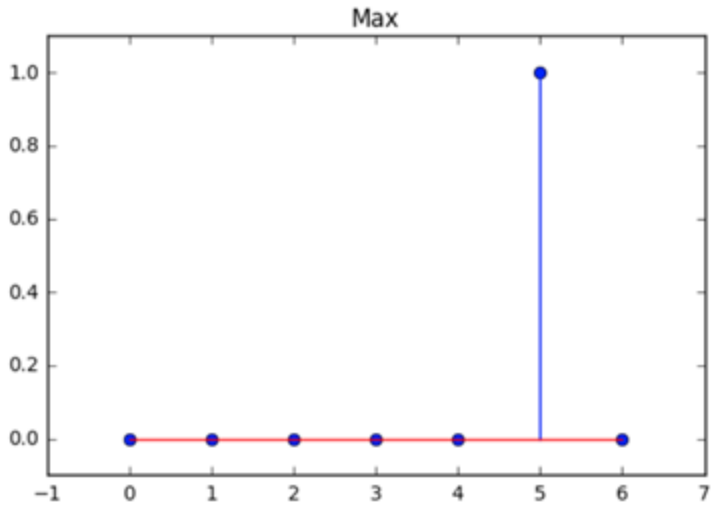


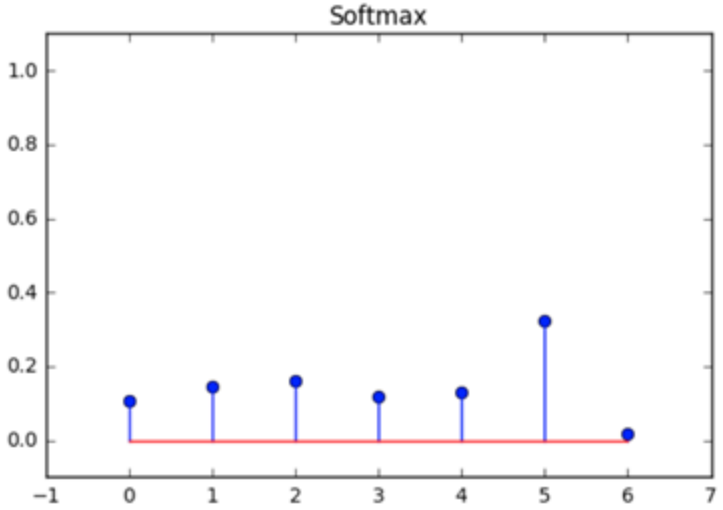
If we know that we only have one possible 'activation' in a given layer, we can use a
'Softmax Activation Layer'
To take advantage of this fact.

$$a_i = \frac{e^{z_i}}{\sum e^{z_j}}$$

Softmax Activation Function







Problems with the max (argmax) function:

1. It is not differentiable
2. It does not give us a probability distribution over all the outputs

$$a_i = \frac{e^{z_i}}{\sum e^{z_j}}$$

$$\frac{\partial E}{\partial z_i} = y - a$$

Softmax Uses

- Basic Use: output layers

- However, the softma layer is used extensively in hidden layers in other more complex architectures. Most notably the softmax layer is used extensively to implement the attention mechanism as used in transformers.

Questions

- Come up with your own examples of multilabel and multiclass classification?
- Can we use multinomial classification for multiple real valued outputs?
- Are softmax output calculations independent of each other?

Other Activation Layers

Hyperbolic Tangent Functions

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = a\ (1 - a)$$

Logistic Activation Function + Derivative

Hyperbolic Tangent Functions

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = a\,(1 - a)$$

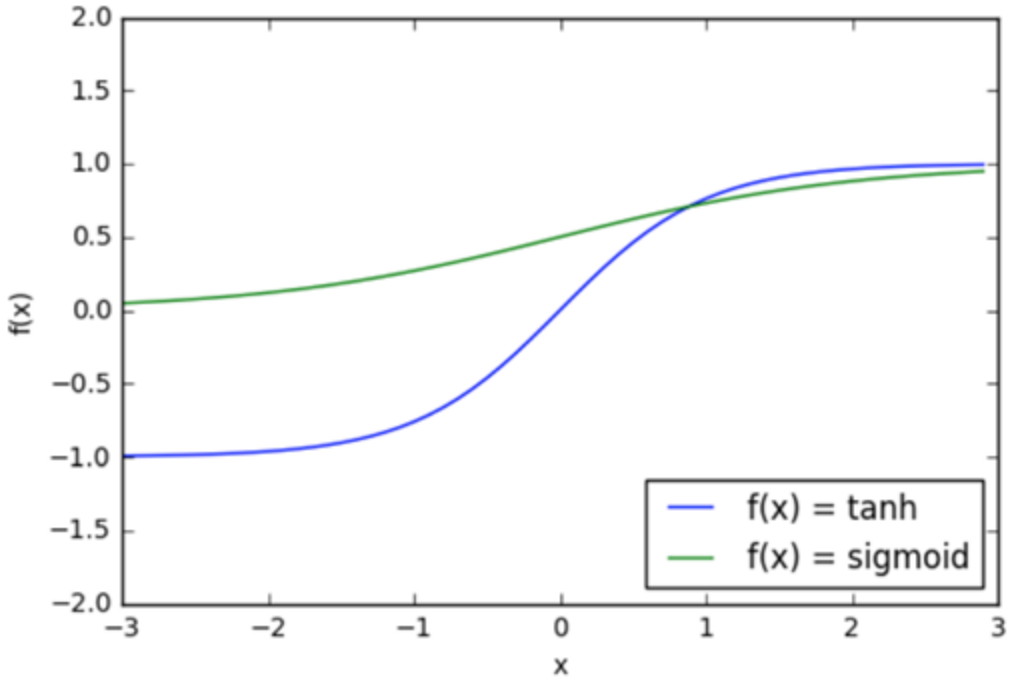
Logistic Activation Function + Derivative

$$g(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

$$g'(z) = 1 - tanh^{2}(z)$$

HTAN Activation Function + Derivative

Hyperbolic Tangent Functions – another sigmoid



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = a\ (1 - a)$$

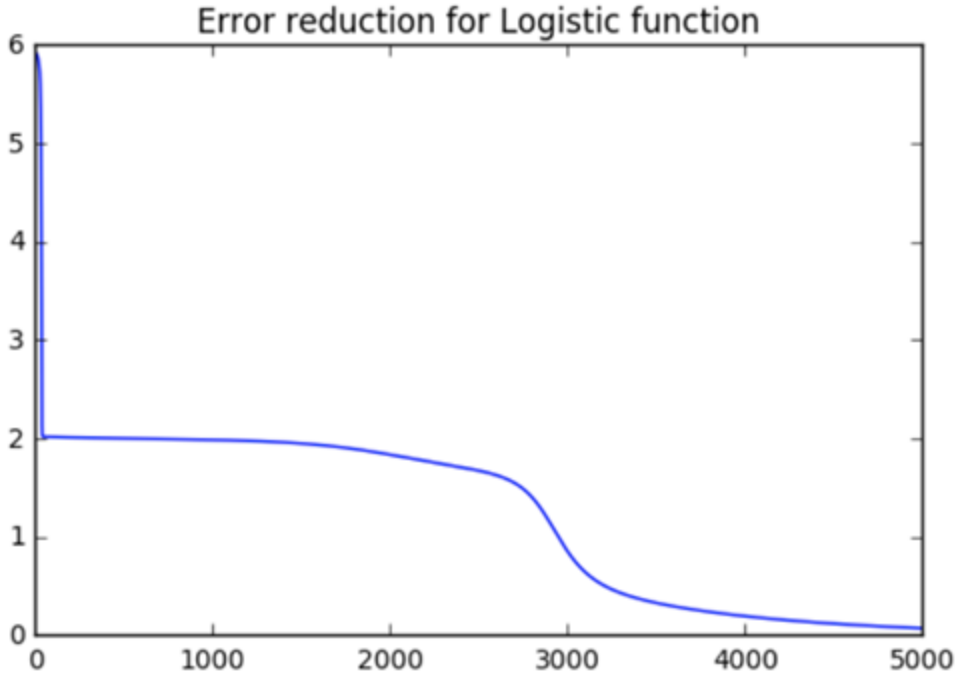
Logistic Activation Function + Derivative

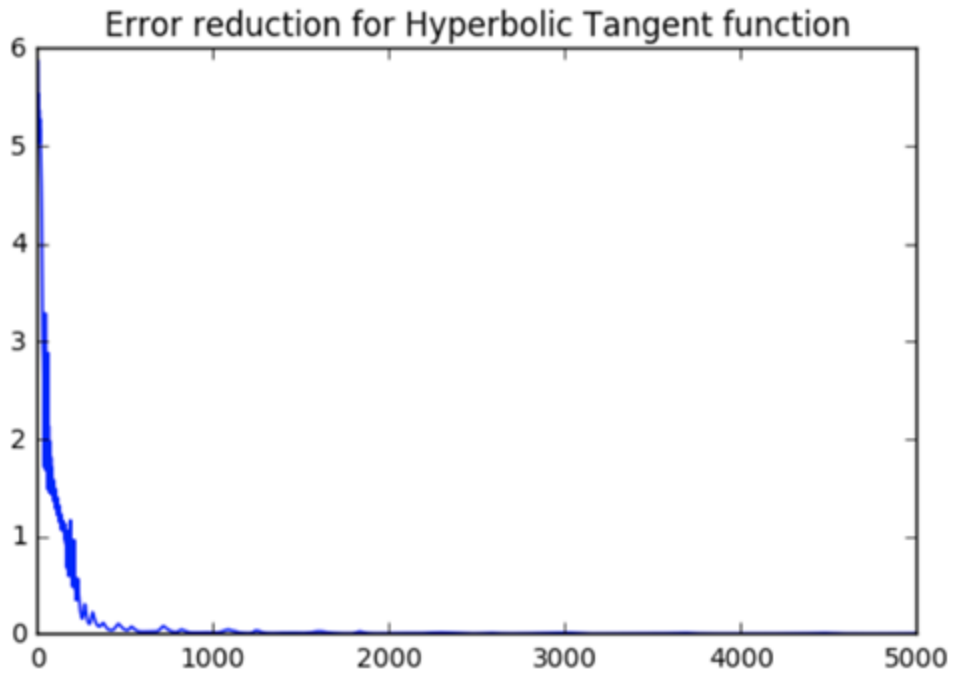
$$g(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

$$g'(z) = 1 - tanh^{2}(z)$$

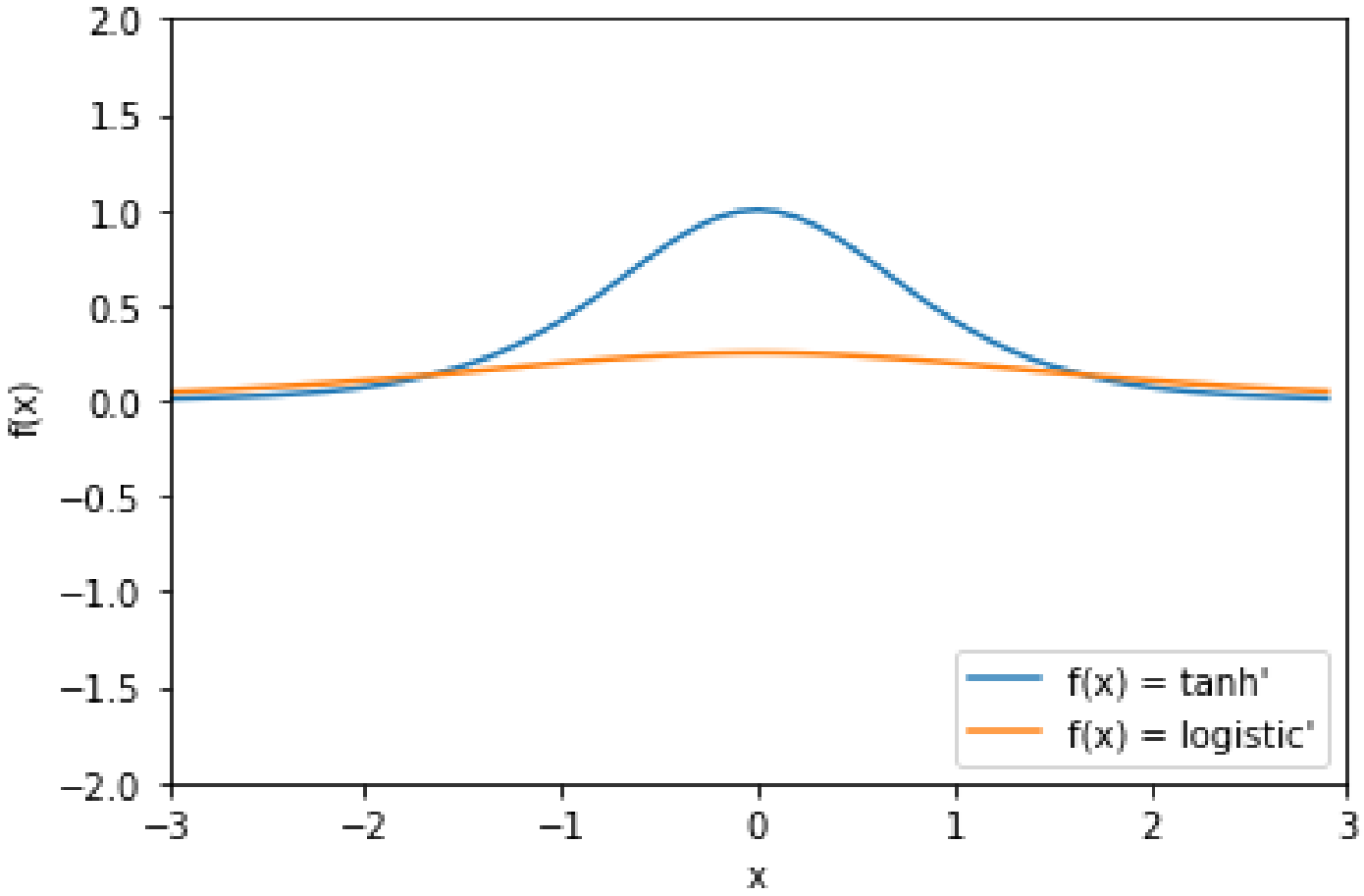
HTAN Activation Function + Derivative

HTAN Performance





HTAN Performance - Cause

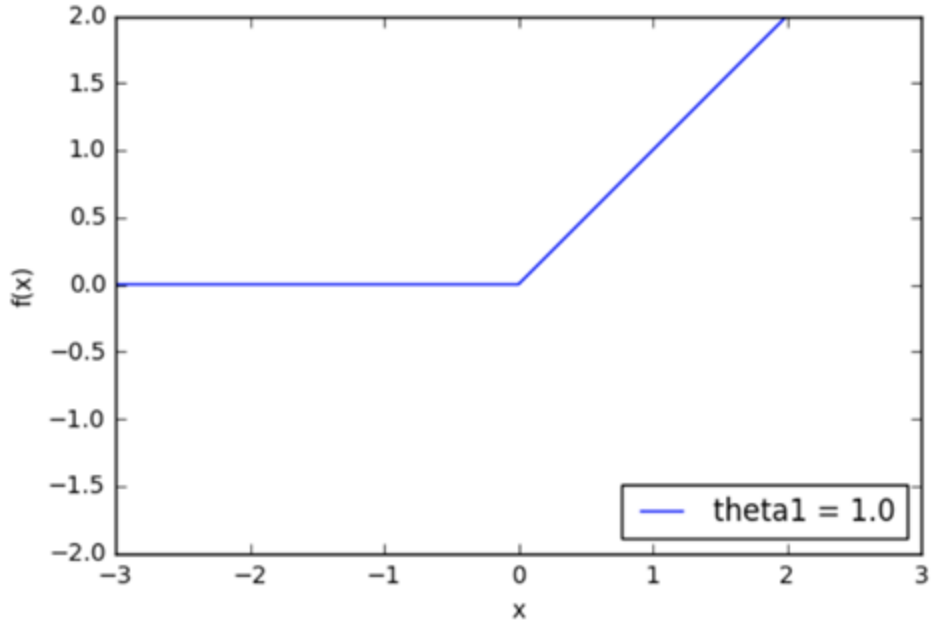


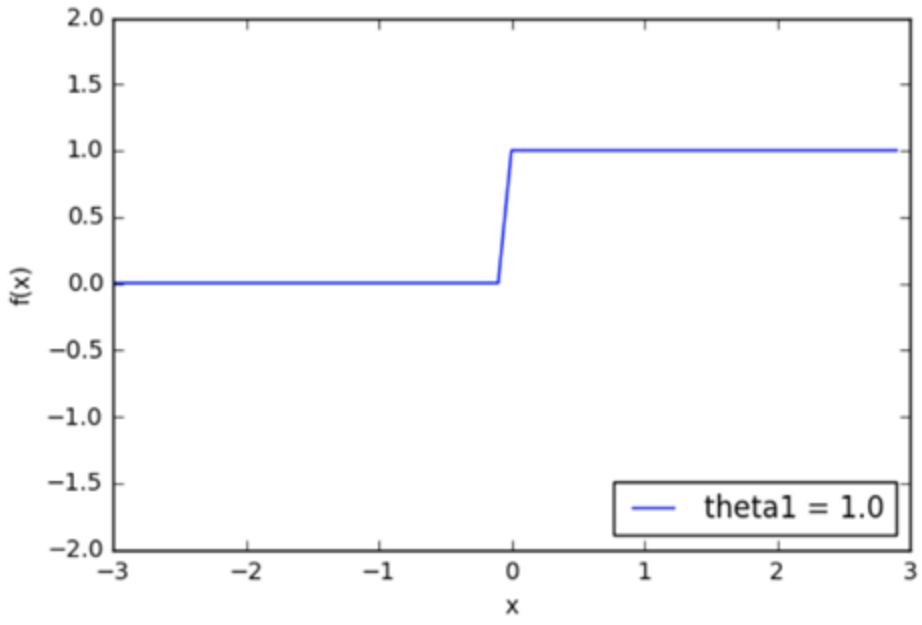
$$g'(z) = a\ (1 - a)$$

$$g'(z) = 1 - tanh^2(z)$$

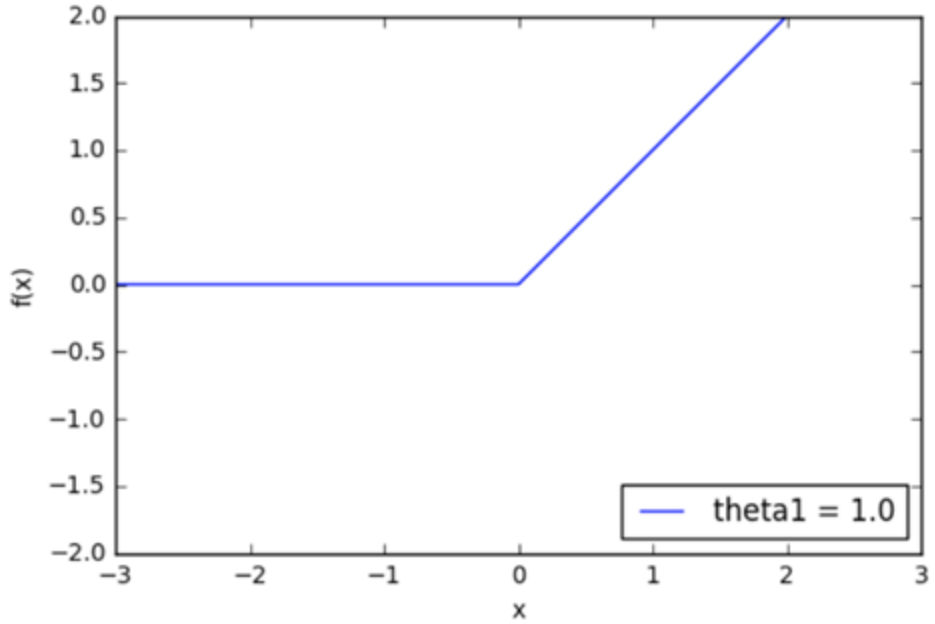
Can we do any better?

Rectified Linear Units

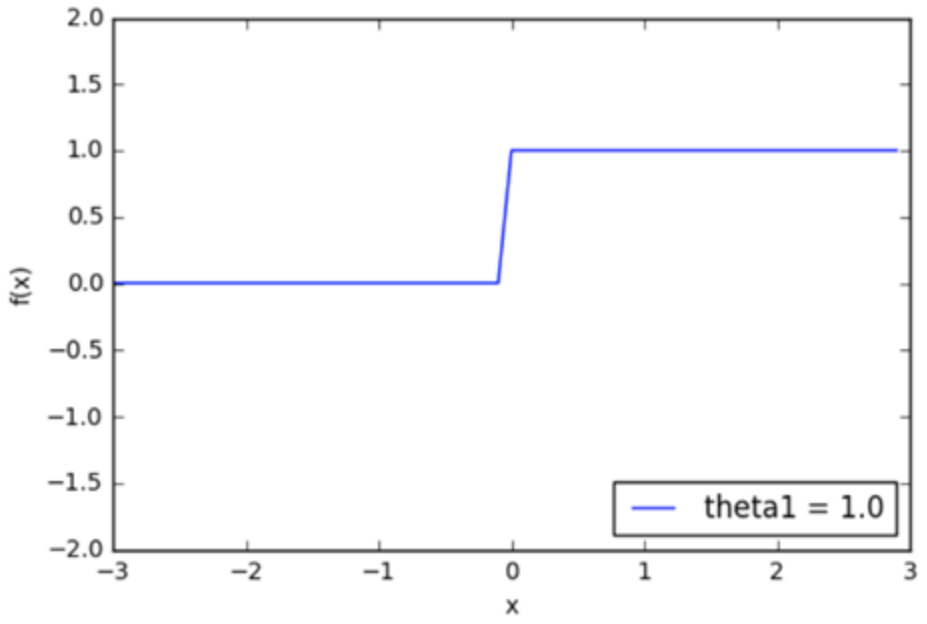




Rectified Linear Units

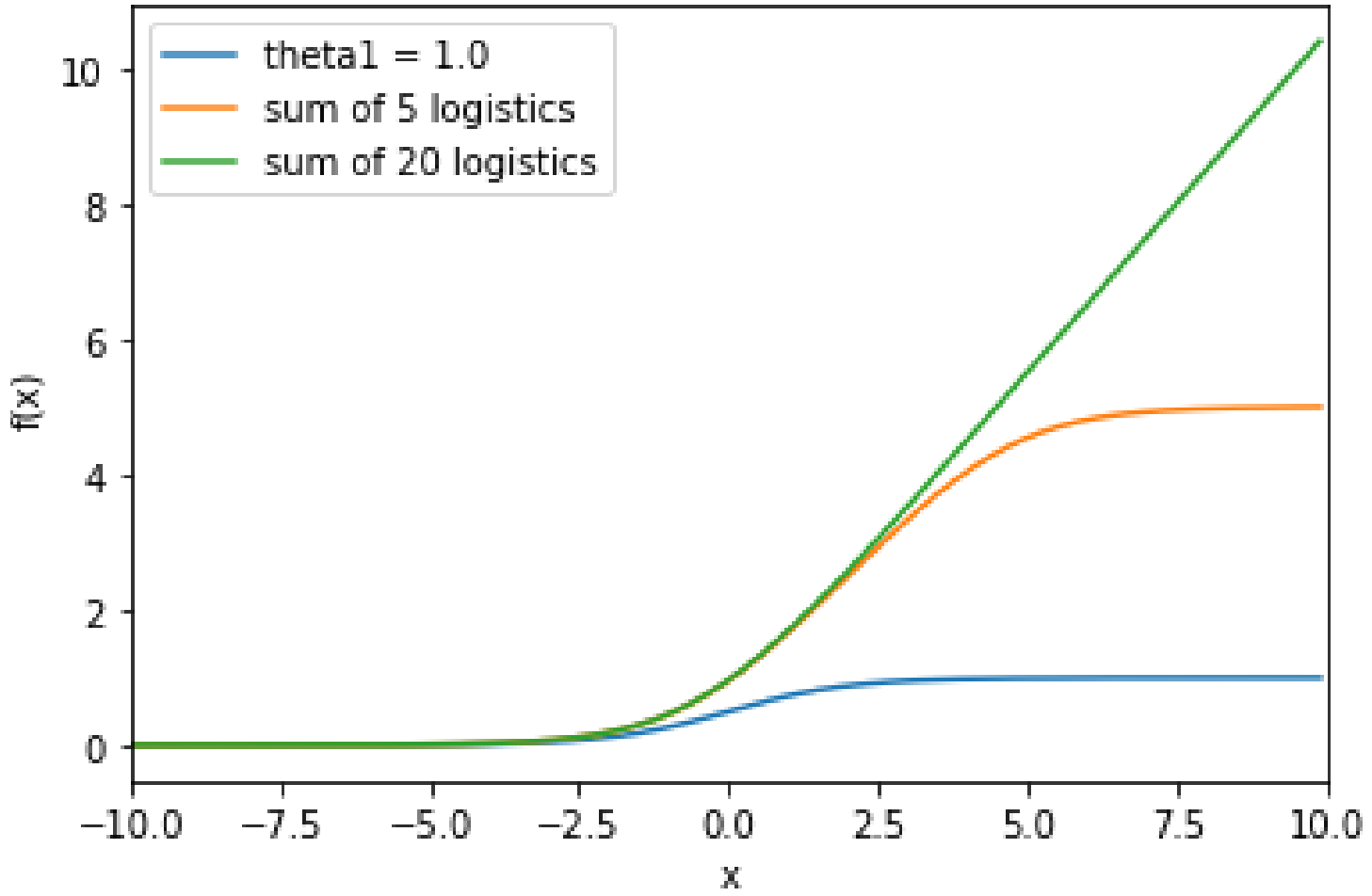


$$g(z) = max(0, z)$$

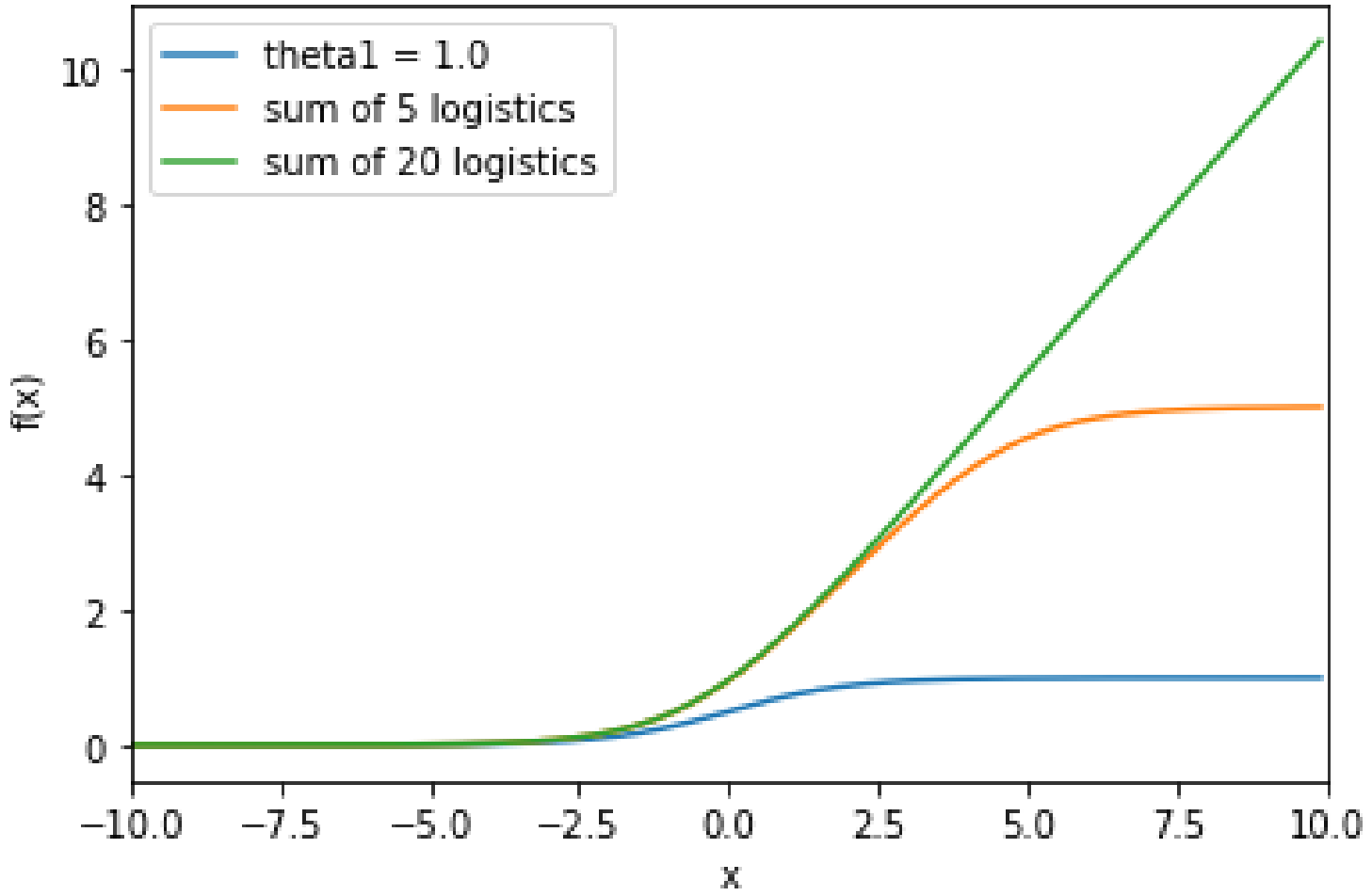


$$g'(z) = \begin{cases} 0 & if\ g(z) <= 0 \\ 1 & if\ g(z) > 0 \end{cases}$$

Rectified Linear Unit Performance Cause



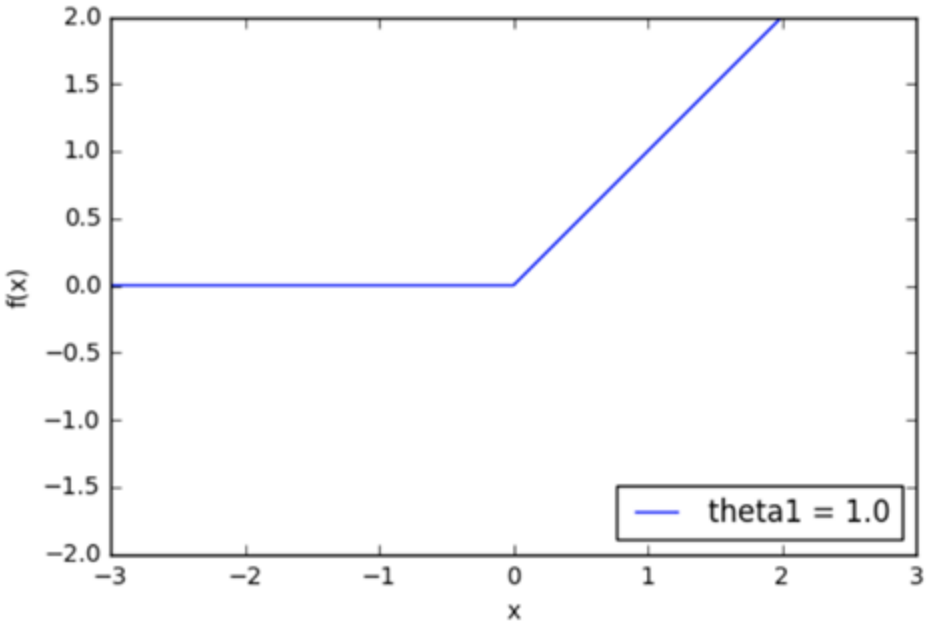
Rectified Linear Unit Performance Cause



Also, the sparse network effecT!!!

Challenges with Rectified Linear Units

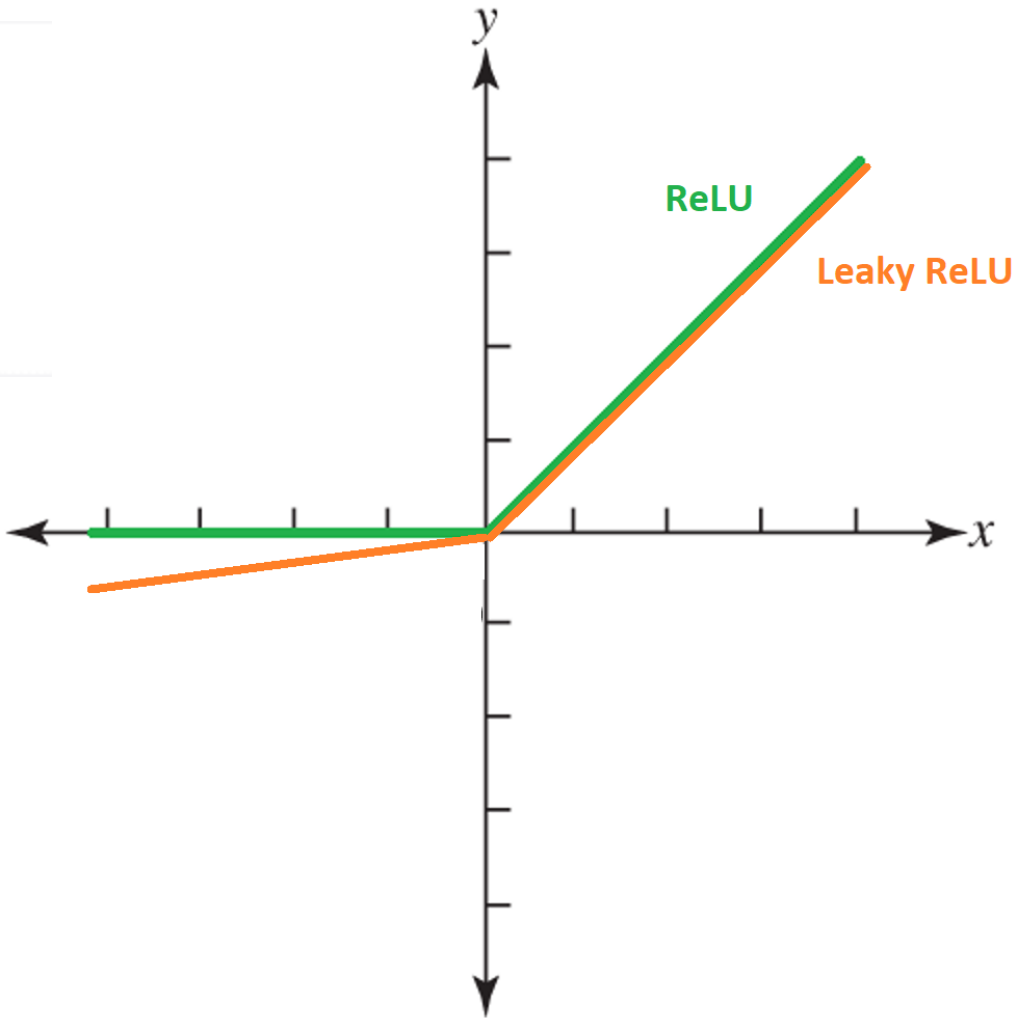
- Dying Relu Problem
- Too many negative outputs can force a relu to produce zero output. Once a zero output has been produced, it is difficult to move from that point.
- Particularly challenging when initialised close to 0 and high learning rates



Beyond Rectified Linear Units – Leaky Relu

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases},$$

- Where alpha is a small number. Typically, 0.01



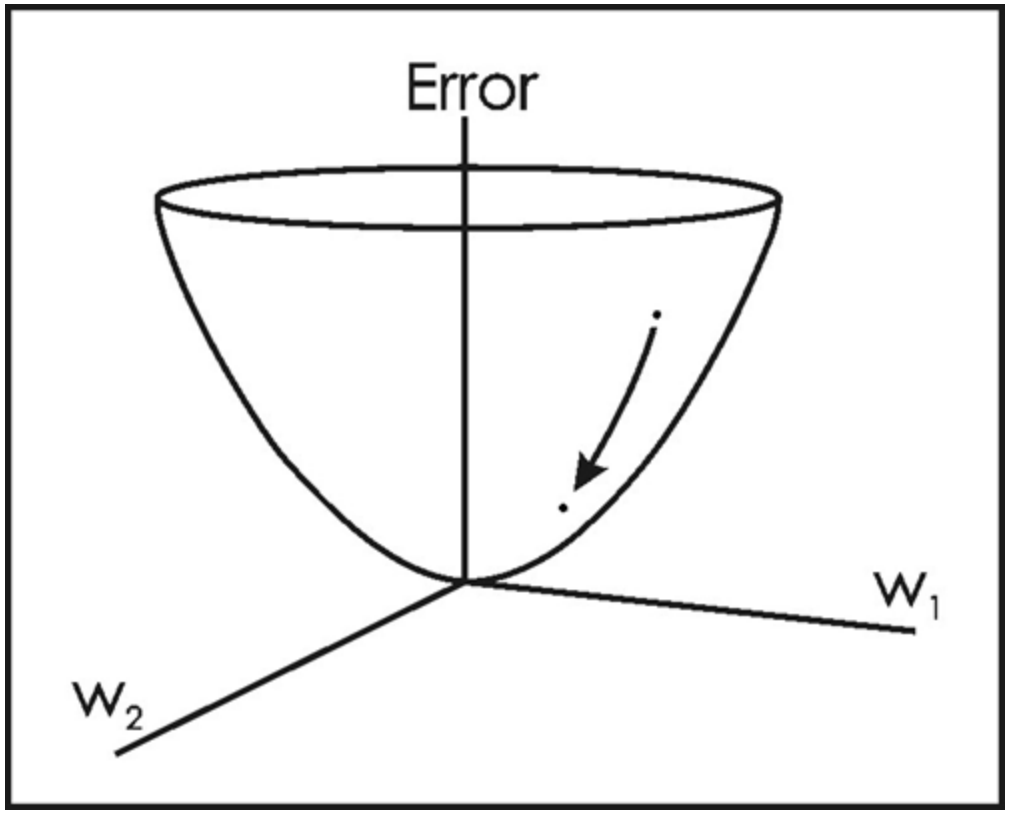
Relu vs Leaky Relu

- Relu is still default. Dying Relu problem better addressed with good initial parameters and (low) careful control of the learning rate

Optimizers for Training

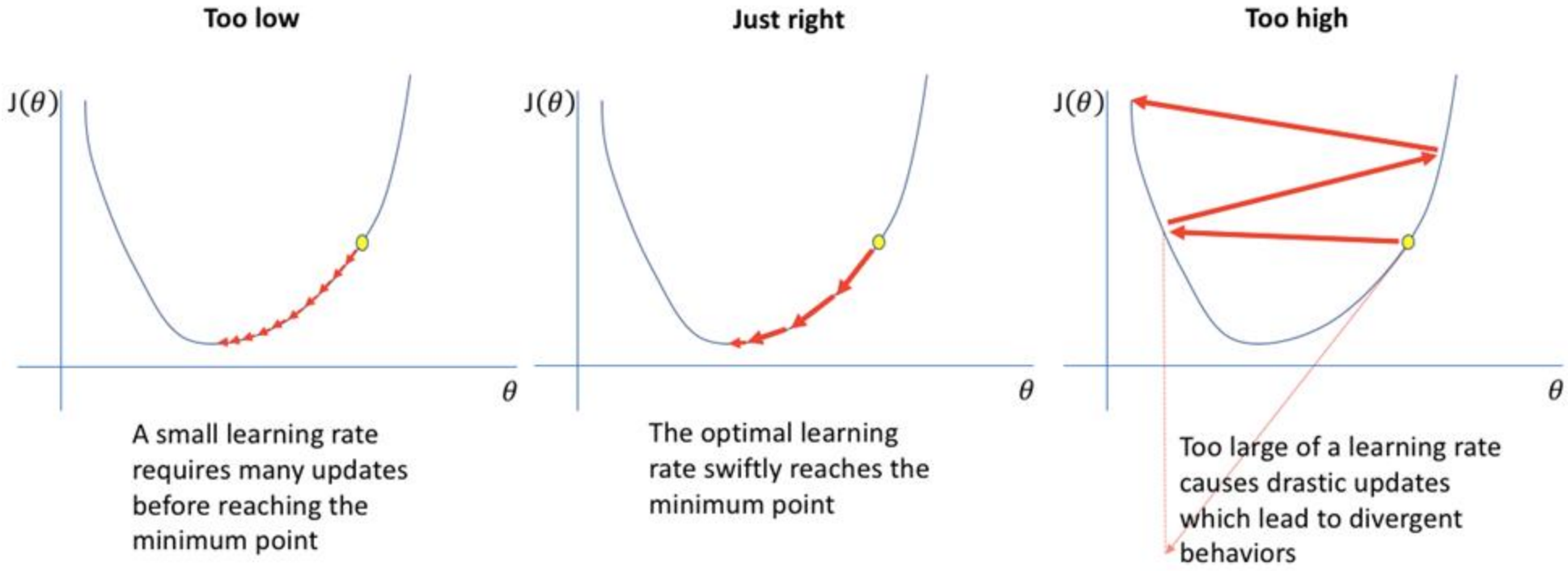
Optimization Methods

Our goal is to reduce the error over time



Problems with Full Gradient Descent 1

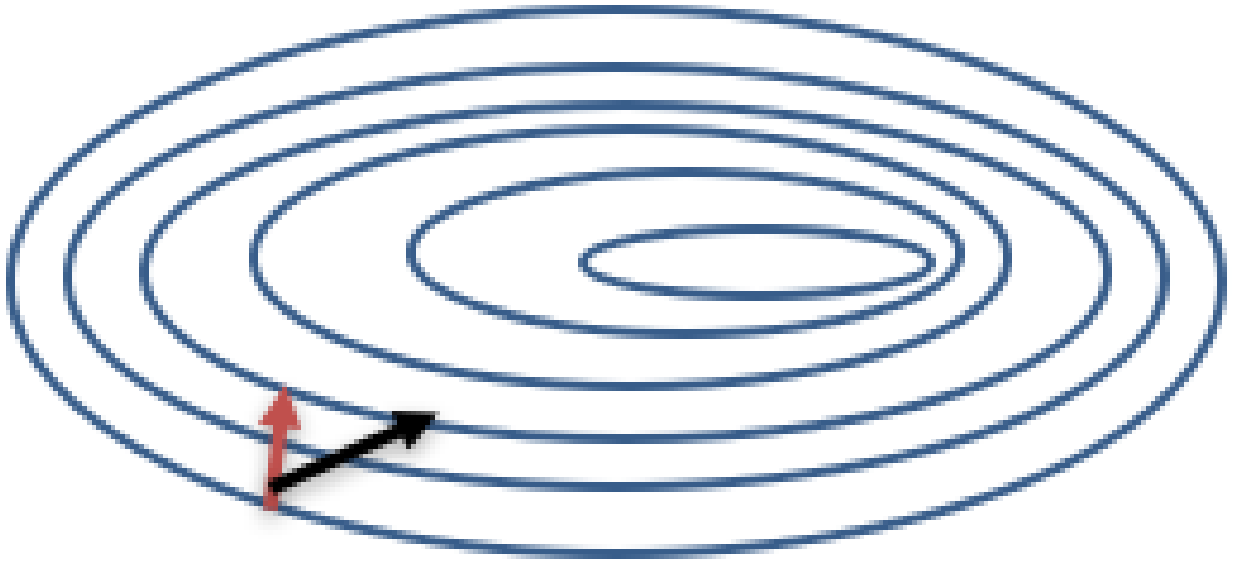
- Selecting a learning rate



https://www.jeremyjordan.me/nn-learning-rate/

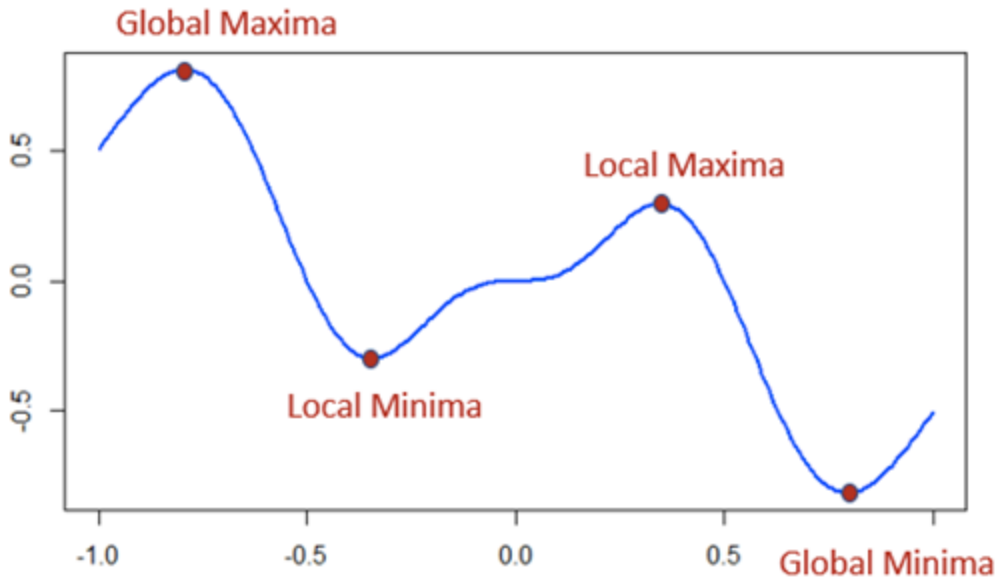
Problems with Full Gradient Descent 1

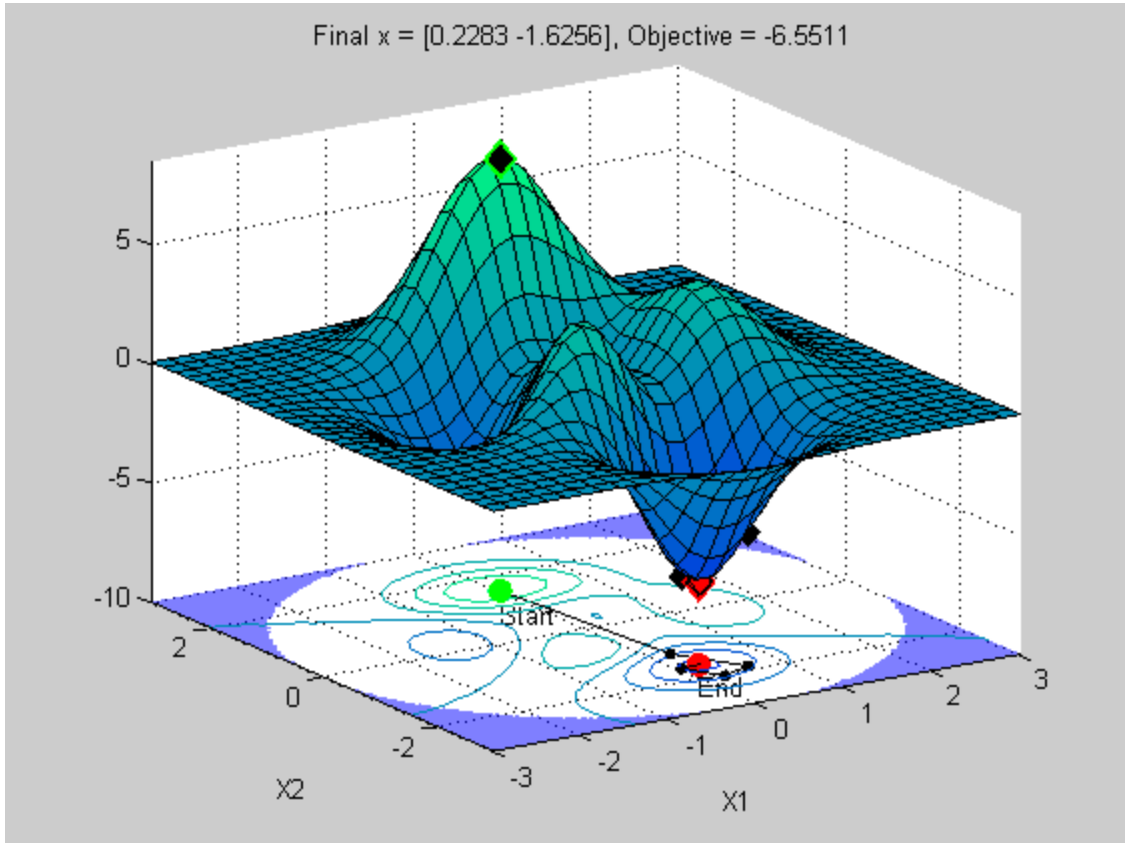
- Meandering towards the minimum



Problems with Full Gradient Descent 2

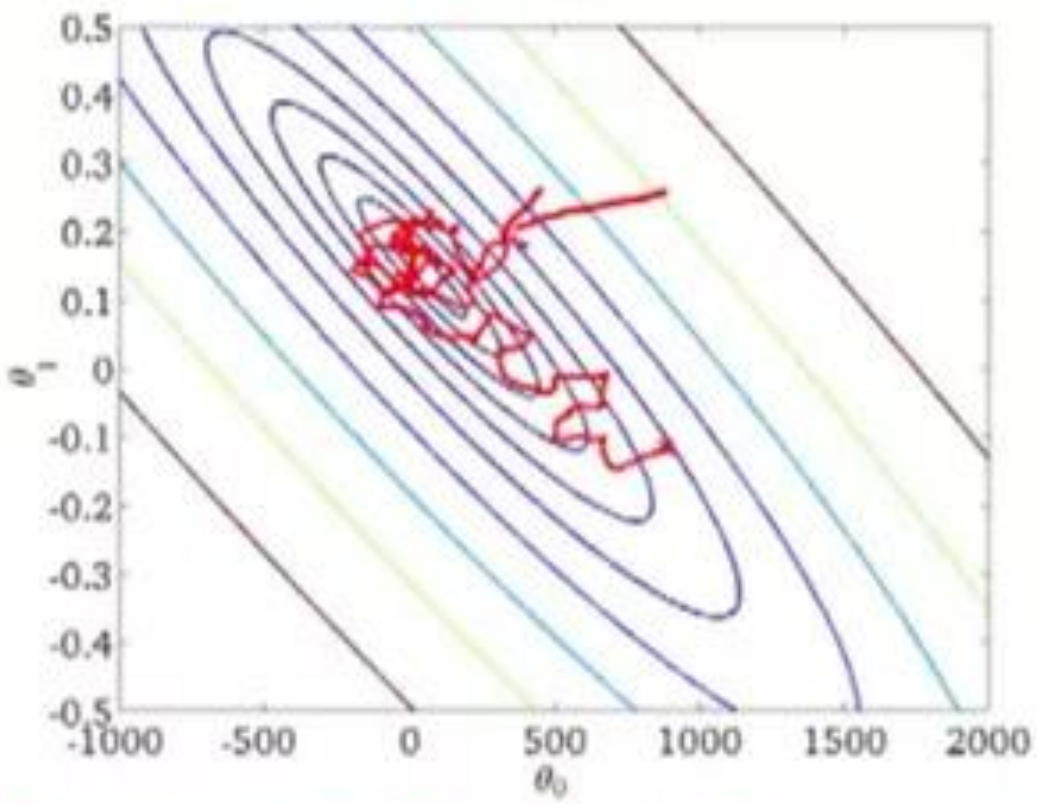
Local minima





Solutions

Stochastic Gradient Descent



Update after each individual example.

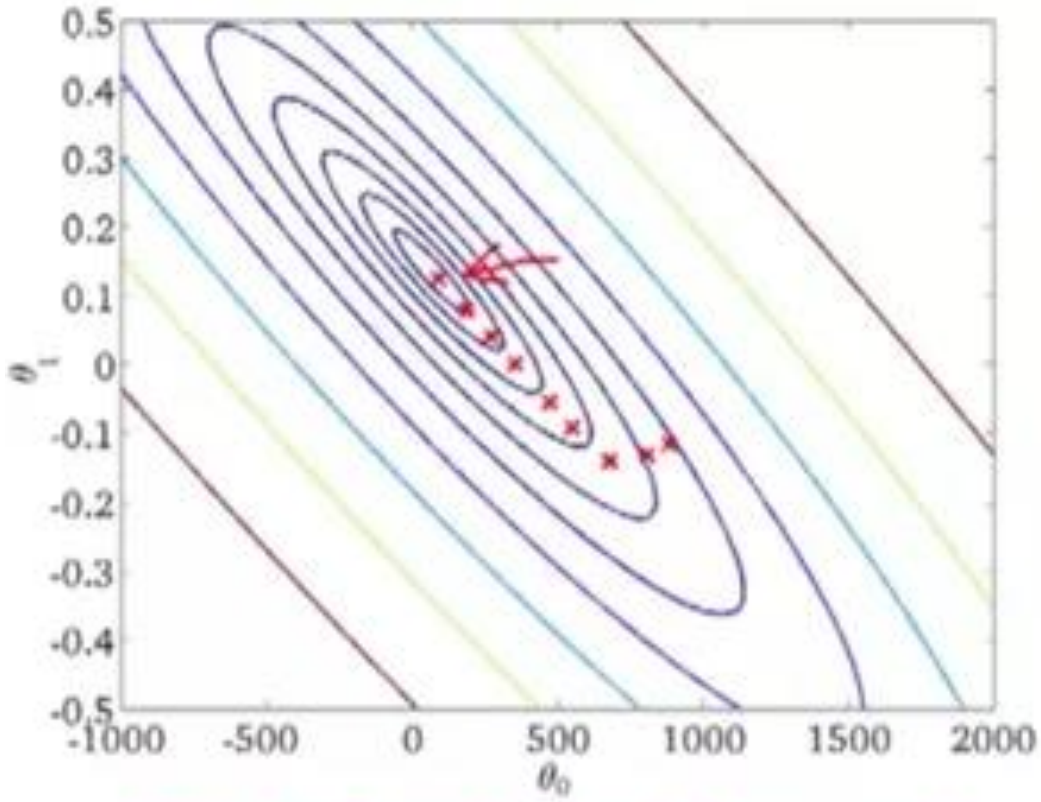
-- this is in contrast to full gradient descent where we look at all of our examples and calculate a total loss before applying gradients

Pros: Convergence is better
Cons: Quite Expensive

Stochastic Gradient Descent (with Batches)

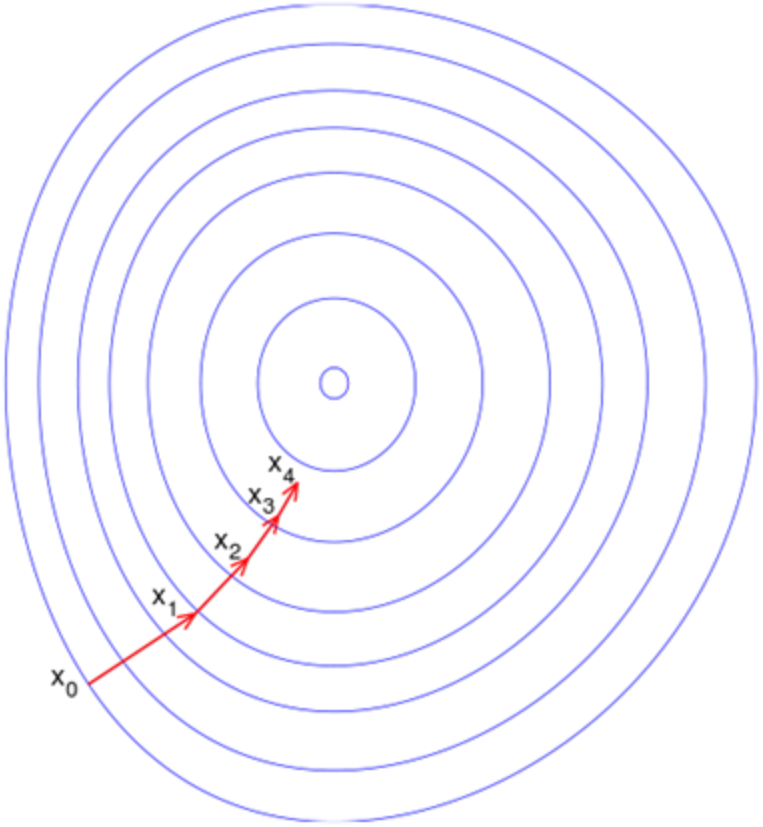
Happy middle ground.

Save up losses from a number of examples (a batch) before applying the updates.



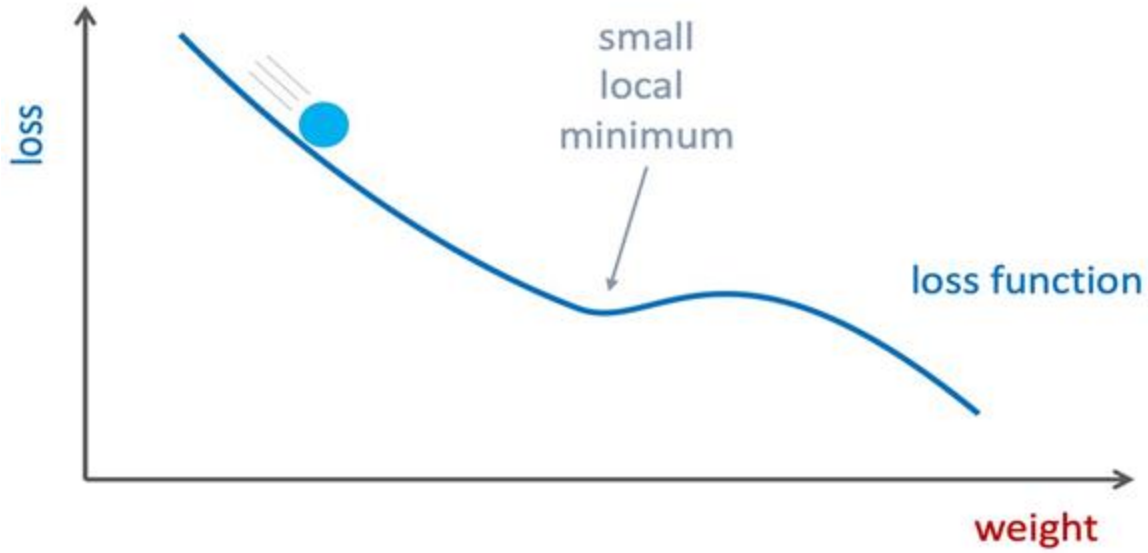
Improving Optimizers

Adjustable Learning Rates



Improving Optimizers

Momentum



momentum

Improving Optimizers

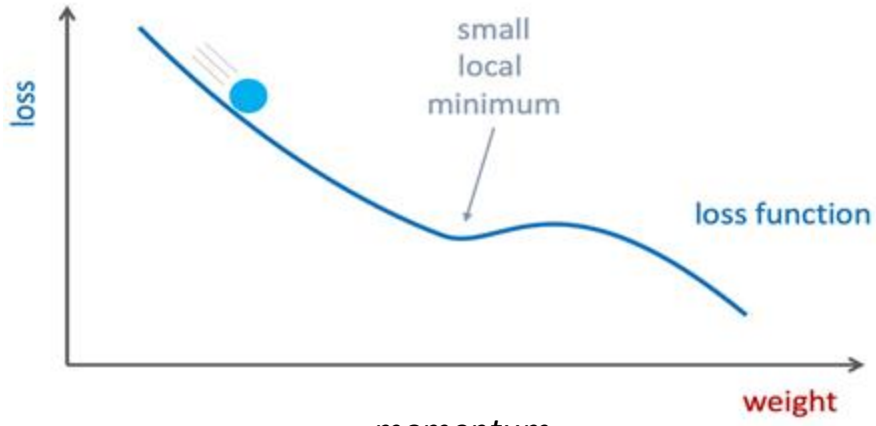
Feature Specific Learning Rates



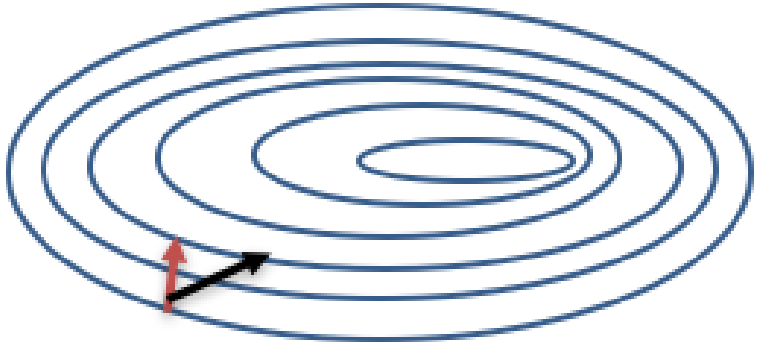
multiple error rates

Improving Optimizers

- Principles
  - Adjust Learning Rate
  - Use some notion of momentum for overcoming local minima
  - Keep a different error rate for each feature (dimension)



momentum

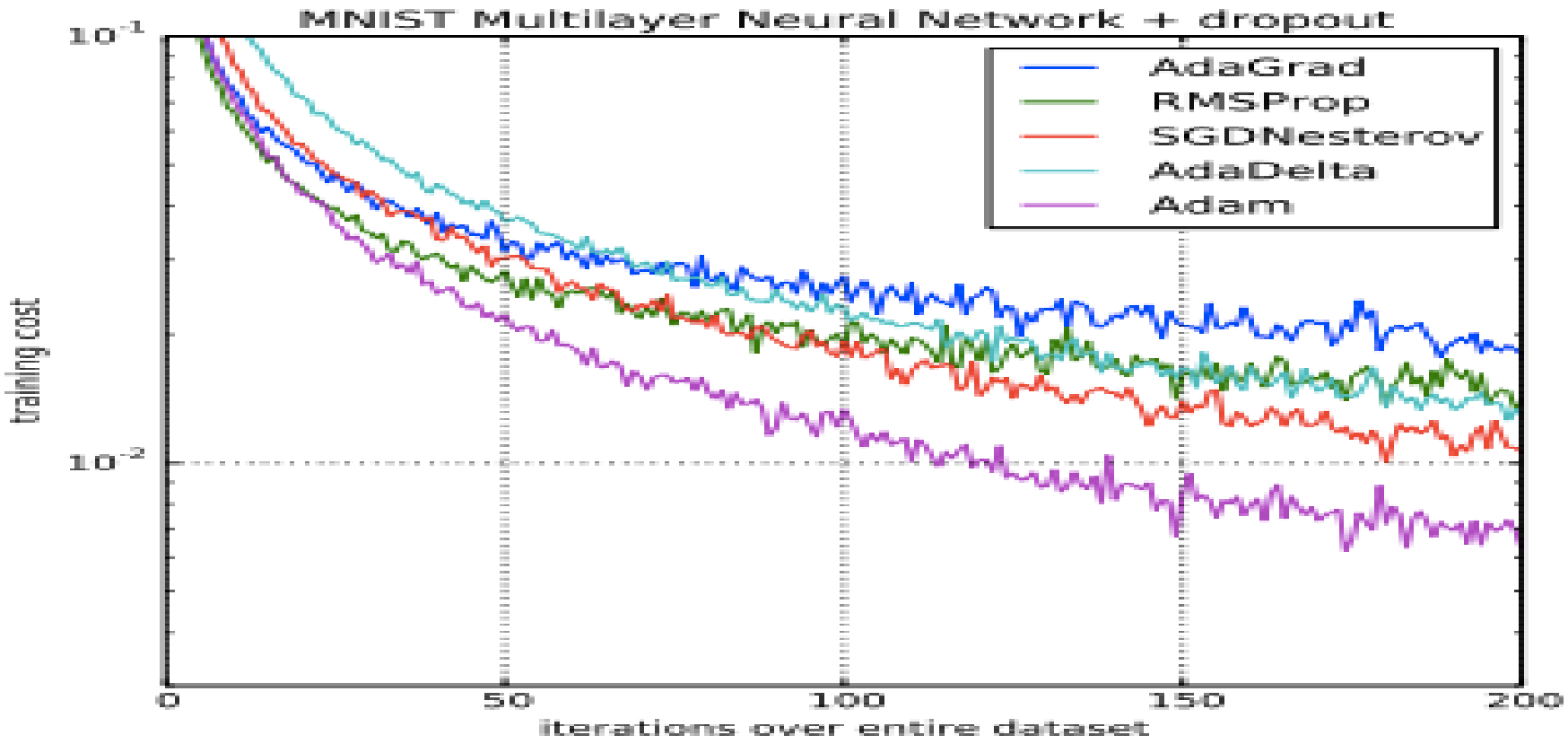


multiple error rates

State of Art Optimizers

- Key Examples
  - Adam
  - Adagrad
  - RMSProp –These are all variations on a theme
    - Using momentum
    - Using multiple learning rates
    - Adjusting learning rates on the fly
- Analytical Approaches (optional)
  - LBFGS – Not seriously used in practical NN applications

Comparing Optimisers



Group Assignment

Split into small groups:

- Within your group split in two, parts A and B. Part A should investigate Leaky RELU while part B should investigate clipping for RELU. Each group take 15 minutes to investigate their variety, and then two minutes each to inform the other half of the team.